Mobile Money Investigations with PostgreSQL and Neo4j

Advanced Big Data Analytics – Project 2 (issued 12 October 2026, due 26 October 2026 23:59 CAT)

Member	Student ID	Main responsibility
<Name 1>	<ID>	<e.g. capture, PostgreSQL, Q1–Q3>
<Name 2>	<ID>	<e.g. Neo4j import, Q4–Q7>
<Name 3>	<ID>	<e.g. benchmarks, load test, report>

GitHub: <URL> ZIP: <file name>. Full outputs, raw repetitions and plans are in results/ of the code submission.

How to use this draft: every number below was produced by the submitted code. PostgreSQL figures come from a reference run (2 vCPU, 8.4 GB RAM, Linux, PostgreSQL 16.15). Re-run the README steps on your own machine, rebuild this file with python report/build_report.py, insert the yellow-box screenshots and fill the orange Neo4j items, then export to PDF. Delete this paragraph before submission.

1 Capture the data

1.1 Workspace

Item	Value
OS	Linux-6.18.44-fc-v140-x86_64-with-glibc2.39
CPU (logical / physical)	x86_64 (2 / 2)
RAM	8.42 GB
Python / numpy	3.13.16 / 2.5.3
PostgreSQL / psycopg	PostgreSQL 16.15 (Ubuntu 16.15-0ubuntu0. / 3.3.6
PostgreSQL memory	shared_buffers 131072 8kB, work_mem 65536 kB, maintenance_work_mem 1048576 kB, effective_cache_size 524288 8kB
PostgreSQL TimeZone	Africa/Kigali
Neo4j / Python driver	<fill: CALL dbms.components()> / <driver>
Neo4j memory	<fill: heap initial/max, page cache from workspace_info.json>

SCREENSHOT TO INSERT: python src/workspace_info.py output plus a successful psql \conninfo and Neo4j Browser/cypher-shell connection; SHOW timezone = Africa/Kigali. Taskbar and clock visible.

1.2 capture.py

capture.py imports the unchanged generator as g, prints dataset_info(), FIELDS and ten records of each stream with itertools.islice, then streams every record through csv.DictWriter (header first, one row at a time). It counts consumed and written rows, logs every million transfers and keeps the transfer audit totals without storing the stream.

Stream	Consumed	Written	Expected	Seconds	CSV GB
customers	228,000	228,000	228,000	0.27	0.0036
accounts	248,000	248,000	248,000	0.34	0.0052
devices	228,600	228,600	228,600	0.25	0.0036
account_devices	285,600	285,600	285,600	0.55	0.0063
transfers	20,003,300	20,003,300	20,003,300	72.95	0.7802
Total	20,993,500		20,993,500	74.36	0.7988

Transfer audit: count 20,003,300; earliest occurred_at 1788235200 (2026-09-01T06:00:00+02:00); latest 1790798399 (2026-09-30T21:59:59+02:00); sum amount_rwf 231,482,327,870 RWF.

SCREENSHOT TO INSERT: python src/capture.py --preview (ten records per stream) and the completed five-stream totals of python src/capture.py.

2 Build PostgreSQL

2.1 Tables and constraints

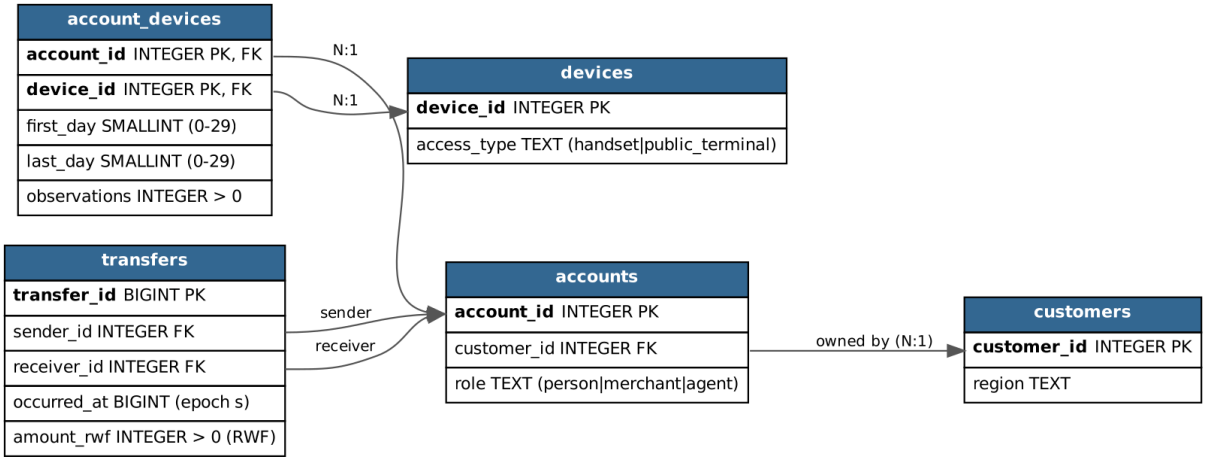


Figure 1 Relational schema (PK bold, FK arrows)

Field	Meaning and unit
customer_id / account_id / device_id	Integer entity identifiers (primary keys)
region	Owner region: Kigali, Northern, Southern, Eastern, Western
role	Wallet role: person, merchant or agent
access_type	handset or public_terminal
first_day, last_day	Inclusive observation day numbers, 0 = 1 Sep 2026 CAT … 29 = 30 Sep; CHECK 0 ≤ first ≤ last ≤ 29
observations	Count of observations of that account on that device within the interval; CHECK > 0
transfer_id	BIGINT event identifier, one row per payment
sender_id, receiver_id	Paying and receiving wallets (both FK to accounts)
occurred_at	BIGINT Unix epoch seconds (UTC instant; displayed in Africa/Kigali, UTC+02:00)
amount_rwf	INTEGER whole Rwandan francs; CHECK > 0. A transfer value, not a balance or fee

All columns are NOT NULL. Primary keys: each entity/event ID, and (account_id, device_id) for account_devices. Foreign keys: accounts.customer_id → customers; account_devices.account_id → accounts; account_devices.device_id → devices; transfers.sender_id and transfers.receiver_id → accounts. Definitions: sql/01_schema.sql and sql/03_constraints_indexes.sql.

SCREENSHOT TO INSERT: \d+ of the five tables and the constraint list from sql/04_checks.sql (convalidated = t for all 16).

2.2 Bulk load and checks

Loading (\copy, parents first) took 12.9 s in total (transfers 12.5 s). Keys, foreign keys, checks, the five required indexes and VACUUM ANALYZE took a further 53.1 s, timed separately (results/pg_load_log.txt, pg_constraints_indexes_log.txt).

Check	Result	Expected

Row counts	228,000 / 248,000 / 228,600 / 285,600 / 20,003,300	capture.py counts
Audit totals	20,003,300; 1788235200–1790798399; 231,482,327,870 RWF	identical
Roles	person 240,000; merchant 6,000; agent 2,000	same
Constraints	16 constraints, all validated	
Sizes (1 GB = 10^9 B)	transfers data 1.2057 GB + indexes 2.4335 GB; database 3.7261 GB	

SCREENSHOT TO INSERT: sql/04_checks.sql output: counts, audit totals, roles, validated constraints, indexes and sizes.

3 Build Neo4j

3.1 Mapping the same facts

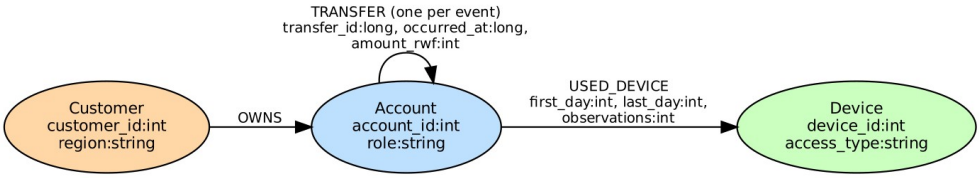


Figure 2 Graph schema

Source field	Graph element
customers.customer_id, region	(:Customer {customer_id:int, region:string})
accounts.account_id, role	(:Account {account_id:int, role:string})
accounts.customer_id	(:Customer)-[:OWNS]->(:Account) via owns.csv (column ignored in Account node file)
devices.device_id, access_type	(:Device {device_id:int, access_type:string})
account_devices.*	(:Account)-[:USED_DEVICE {first_day, last_day, observations}]->(:Device)
transfers.*	(:Account)-[:TRANSFER {transfer_id:long, occurred_at:long, amount_rwf:int}]->(:Account), one per event

3.2 Import

prepare_import.py writes temporary copies with typed headers (data bytes streamed, sources unchanged) and derives owns.csv in (customer_id, account_id) order. Headers: customer_id:ID(Customer),region:string | account_id:ID(Account),:IGNORE,role:string | device_id:ID(Device),access_type:string | :START_ID(Customer),:END_ID(Account) | :START_ID(Account),:END_ID(Device),first_day:int,last_day:int,observations:int | transfer_id:long,:START_ID(Account),:END_ID(Account),occurred_at:long,amount_rwf:int.

```
neo4j-admin database import full neo4j --overwrite-destination=true --id-type=integer --
nodes=Customer=customers.csv --nodes=Account=accounts.csv --nodes=Device=devices.csv --
relationships=OWNS=owns.csv --relationships=USED_DEVICE=used_device.csv --
relationships=TRANSFER=transfers.csv
```

FILL FROM YOUR NEO4J RUN: import seconds; constraint/index setup seconds; store GB (data/databases/neo4j) and transaction-log GB (data/transactions/neo4j).

Check (cypher/03_checks.cypher)	Expected	Neo4j result
Customer / Account / Device nodes	228,000 / 248,000 / 228,600	
Total nodes	704,600	

OWNS / USED_DEVICE / TRANSFER	248,000 / 285,600 / 20,003,300	
Total relationships	20,536,900	
Roles	person 240,000, merchant 6,000, agent 2,000	
Accounts without / with >1 owner	0 / 0	
Distinct transfer_id	20,003,300	
Audit totals	1788235200, 1790798399, 231,482,327,870	

SCREENSHOT TO INSERT: import completion summary; SHOW CONSTRAINTS / SHOW INDEXES (all ONLINE); 03_checks results; then prepare_import.py --clean.

3.3 One account in both models

Account 998 (person) is owned by Customer 998 (Kigali), observed on handset 998 (days 0–29, 39 observations) and its first outgoing transfer is #113540 to merchant 240004 on 1 Sep 08:43:29 CAT for 2,100 RWF (src/trace_account.py). In SQL each step is a JOIN on a foreign key (accounts.customer_id, account_devices.account_id/device_id, transfers.sender_id); in Neo4j the same foreign keys are stored as relationships (OWNS, USED_DEVICE, TRANSFER) that are followed directly from node to node. The account_devices table and the transfers table become relationships with properties. Account 998 paid account 1111 nine separate times (and 63 seven times). Collapsing repeats into one edge would erase each transfer_id, its timestamp and amount, so Q1 counts, Q2 daily totals, the strict time order and 600 s gaps of Q5 and the amount rule of Q5/Q6 could no longer be evaluated.

SCREENSHOT TO INSERT: trace_account.py output: SQL JOIN row and Cypher MATCH row side by side (identical values).

4 Routine business questions

Hour windows: 09:45:00 inclusive to 10:45:00 exclusive CAT, converted in Python (queries.hour_bounds). All queries are in src/queries.py, with readable copies in sql/queries and cypher/queries.

4.1 Q1 Account activity

Set	Account	Date	Outgoing count	Total RWF	SQL = reference
A	998	6 Sep	1	200,000	MATCH
B	147228	13 Sep	1	1,050,000	MATCH
C	206978	8 Sep	1	300,000	MATCH
D	237998	6 Sep	1	45,000	MATCH
E	144228	13 Sep	1	40,000	MATCH
F	203978	8 Sep	1	30,000	MATCH

Outgoing value is not a balance: it ignores incoming transfers, the opening balance, cash-in/cash-out at agents, fees and reversals, none of which are in these records.

SCREENSHOT TO INSERT: SQL and Cypher Q1 results for all six sets (one consolidated screenshot).

4.2 Q2 Daily activity

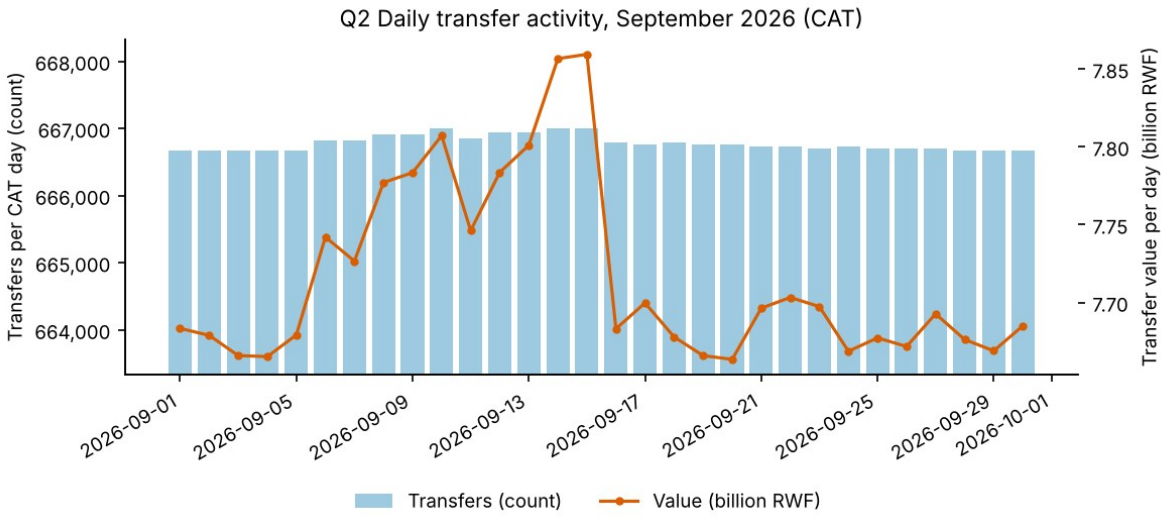


Figure 3 Daily transfer count and value (CAT days)

30 days returned; counts sum to 20,003,300 and values to 231,482,327,870 RWF, reconciling exactly with the capture audit. Pattern: count is almost flat (≈666,667 per day from 06:00 to 22:00 CAT), but 6–15 September carry ~150–330 extra transfers per day and up to +190 million RWF (14–15 Sep 7.86 bn vs about 7.67–7.70 bn on other days). These extra large-value transfers are the scenario payments examined in Section 5. Transfer value is not provider revenue: revenue comes from fees, and value counts money that merely moves (and may move several times) between wallets.

SCREENSHOT TO INSERT: Q2 SQL and Cypher results (30 rows), the reconciliation check and the chart.

4.3 Q3 Shared devices

Set	Account	Rows	other account @ device [overlap days] (all handsets)
A	998	4	16836@228001 [0,29]; 220998@998 [0,29]; 24755@228001 [0,29]; 8917@228001 [0,29]
B	147228	9	147226@147226 [0,29]; 147227@147226 [0,29]; 147229@147226 [0,29]; 147230@147226 [0,29]; 155147@228018 [0,29]; 163066@228018 [0,29]; 170985@228018 [0,29]; 178904@228018 [0,29]; 186823@228018 [0,29]
C	206978	9	206976@206976 [11,29]; 206977@206976 [0,29]; 206979@206976 [0,29]; 206980@206976 [0,29]; 214897@228043 [0,29]; 222816@228043 [0,29]; 230735@228043 [0,29]; 238654@228043 [0,29]; 6573@228043 [0,29]
D	237998	4	13836@228301 [0,29]; 17998@17998 [0,29]; 21755@228301 [0,29]; 5917@228301 [0,29]
E	144228	9	144226@144226 [0,29]; 144227@144226 [0,29]; 144229@144226 [0,29]; 144230@144226 [0,29]; 152147@228318 [0,29]; 160066@228318 [0,29];

			167985@228318 [0,29]; 175904@228318 [0,29]; 183823@228318 [0,29]
F	203978	9	203976@203976 [11,29]; 203977@203976 [0,29]; 203979@203976 [0,29]; 203980@203976 [0,29]; 211897@228343 [0,29]; 219816@228343 [0,29]; 227735@228343 [0,29]; 235654@228343 [0,29]; 3573@228343 [0,29]

Legitimate sharing: account 998 and account 220998 share handset 998 for days 0–29 because both wallets belong to Customer 998 (a secondary wallet on the owner's phone); B's handset 147226 is a household phone also used by accounts 147226–147230. Overlap only means both accounts were observed on the device at some point within the overlapping days, not at the same moment: intervals [2,7] and [5,11] overlap on [5,7] even if one person used the phone in the morning and another at night.

SCREENSHOT TO INSERT: Q3 results/counts for all six sets in both engines.

5 Connected accounts

5.1 Q4 Ownership and device network

Set	Links	Accounts	By owner region
A	4	10	Kigali 3, Northern 7
A	6	43	Eastern 6, Kigali 9, Northern 13, Southern 9, Western 6
A	8	175	Eastern 30, Kigali 33, Northern 37, Southern 45, Western 30
A	10	187	Eastern 36, Kigali 35, Northern 37, Southern 45, Western 34
B	4	46	Eastern 13, Kigali 6, Northern 7, Southern 9, Western 11
B	6	178	Eastern 37, Kigali 30, Northern 35, Southern 45, Western 31
B	8	188	Eastern 40, Kigali 33, Northern 35, Southern 47, Western 33
B	10	228	Eastern 45, Kigali 41, Northern 46, Southern 57, Western 39
C	4	146	Eastern 23, Kigali 24, Northern 29, Southern 43, Western 27
C	6	483	Eastern 82, Kigali 90, Northern 97, Southern 121, Western 93
C	8	832	Eastern 151, Kigali 172, Northern 167, Southern 186, Western 156
C	10	1433	Eastern 278, Kigali 293, Northern 295, Southern 294, Western 273
D	4	10	Eastern 2, Northern 1, Western 7
D	6	44	Eastern 8, Kigali 10, Northern 7, Southern 6, Western 13
D	8	180	Eastern 32, Kigali 50, Northern 31, Southern 30, Western 37
D	10	192	Eastern 34, Kigali 50, Northern 37, Southern 34, Western 37
E	4	47	Eastern 6, Kigali 10, Northern 13, Southern 11, Western 7
E	6	183	Eastern 30, Kigali 50, Northern 37, Southern 31, Western 35
E	8	193	Eastern 33, Kigali 52, Northern 40, Southern 33, Western 35
E	10	230	Eastern 40, Kigali 62, Northern 45, Southern 39, Western 44
F	4	147	Eastern 25, Kigali 44, Northern 29, Southern 27, Western 22

F	6	488	Eastern 93, Kigali 125, Northern 92, Southern 96, Western 82
F	8	837	Eastern 161, Kigali 190, Northern 164, Southern 174, Western 148
F	10	1426	Eastern 278, Kigali 302, Northern 295, Southern 288, Western 263

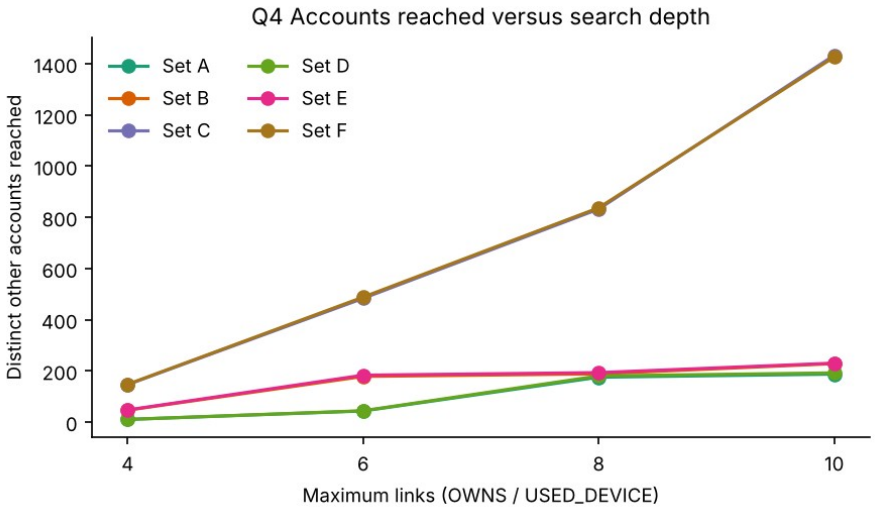


Figure 4 Accounts reached versus depth

[Record the prediction your group made after seeing depths 4 and 6, BEFORE running 8 and 10, and compare it here.]

Measured totals: A 10→43→175→187, B 46→178→188→228, C 146→483→832→1,433, D 10→44→180→192, E 47→183→193→230, F 147→488→837→1,426. A, B, D and E grow sharply once and then level off; C and F keep growing roughly threefold every two links because their early neighbourhood already includes a household phone and a replacement handset, each opening further households. Supporting path (Figure 5): Account 998 – Device 228001 – Account 24755 – Device 24751 – Account 24751 (four links). Deeper links add weak associations: by 8–10 links an account is connected only because a phone was shared by a household member of someone who shared a different phone with a third person; no single observation links it to the review account.

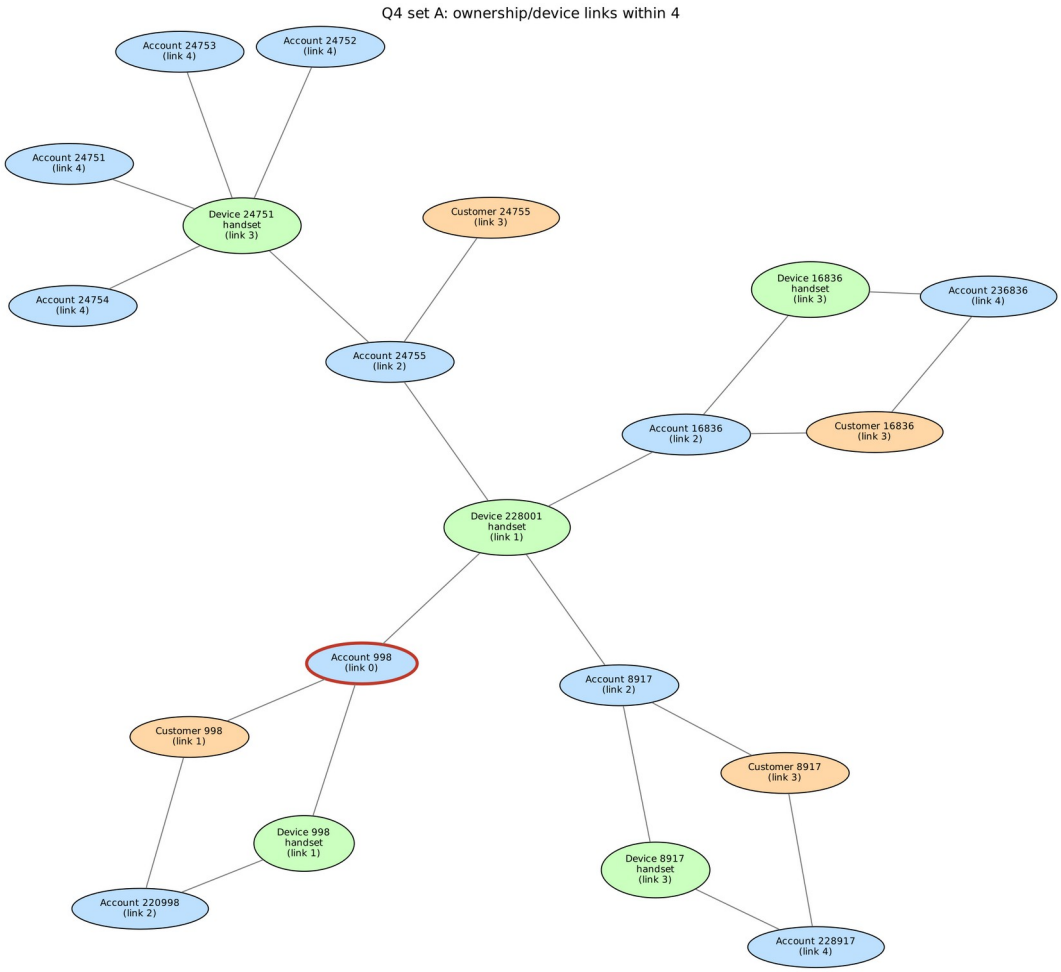


Figure 5 Ownership/device network of set A within four links

SCREENSHOT TO INSERT: all 24 Q4 comparisons (SQL vs Cypher), the plot and one supporting path from Neo4j (Q4_PATH_CYPHER in queries.py).

5.2 Q5 Rapid onward payments

Set	Chains	(t1, t2, t3, final receiver)
A	1	(20000001, 20000002, 20000003, 24755)
B	1	(20000092, 20000093, 20000094, 170985)
C	1	(20000229, 20000230, 20000231, 230735)
D	0	none
E	0	none
F	0	none

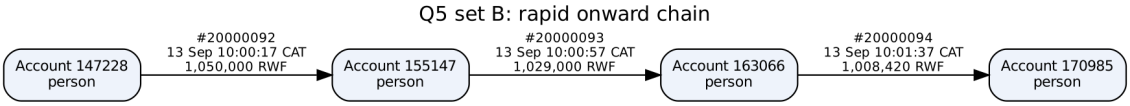


Figure 6 Set B chain

Example (B): #20000092 147228→155147 at 13 Sep 10:00:17 CAT for 1,050,000 RWF; #20000093 155147→163066 at 10:00:57 for 1,029,000 (98%); #20000094 163066→170985 at 10:01:37 for 1,008,420 (98%). The chain does not prove the same funds moved: wallets are fungible, each intermediary may already have held enough balance, and similar amounts

40 s apart can be coincidental or ordinary (e.g. paying a supplier after being paid). Without balances, the data cannot trace specific money.

SCREENSHOT TO INSERT: six matching Q5 counts in both engines and the B chain with IDs, CAT times and amounts.

5.3 Q6 Circular payments

Set	Cycles	Transfer-ID sequence
A	1	[20000001, 20000002, 20000003, 20000004] (len 4)
B	0	none
C	1	[20000229, 20000230, 20000231, 20000232, 20000233, 20000234] (len 6)
D	0	none
E	0	none
F	0	none

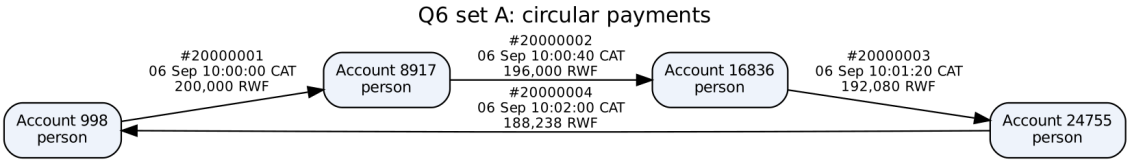


Figure 7 Set A cycle

A: 998→8917→16836→24755→998 on 6 Sep between 10:00:00 and 10:02:00 CAT, 200,000 → 196,000 → 192,080 → 188,238 RWF. C: a six-transfer cycle 206978→…→6573→206978 on 8 Sep 10:00:42–10:04:02, 300,000 falling 2% per hop to 271,176 RWF. Further evidence to request: wallet balances before and after, KYC/identity and relationship of the four owners, the device and location that initiated each payment, fees charged, any cash-out at agents afterwards and the history of this group in other months.

SCREENSHOT TO INSERT: six Q6 counts in both engines and one cycle.

5.4 Q7 Shortest transfer connection

Set	From	To	Bound 4	Bound 6	One supporting path (SQL)	Check
A	998	24755	3	3	998 → 8917 → 16836 → 24755	valid
B	147228	186823	no route ≤4	5	147228 → 147793 → 147197 → 148101 → 178904 → 186823	valid
C	206978	6573	no route ≤4	5	206978 → 214897 → 222816 → 230735 → 238654 → 6573	valid
D	237998	21755	3	3	237998 → 5917 → 13836 → 21755	valid
E	144228	183823	no route ≤4	5	144228 → 152147 → 160066 → 167985 → 175904 → 183823	valid
F	203978	3573	no route ≤4	5	203978 → 211897 → 219816 → 227735 → 235654 → 3573	valid

Every path was checked hop by hop: a stored transfer exists in that direction (example transfer_id recorded), both accounts are persons, the hops are contiguous and their number equals the minimum (results/q7_path_checks.json). B's SQL path (147228→147793→147197→148101→178904→186823) differs from the scenario route but is equally short,

illustrating that shortest paths are not unique. No route within four transfers only means no connection that short among person accounts; longer routes, routes through merchants/agents, cash, or other providers are not excluded. A structural route ignores time and amount, so its transfers may run backwards in time (as in the brief's teaching example) and need not carry the same funds.

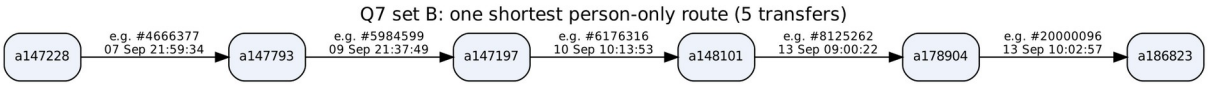


Figure 8 One shortest person-only route for set B

SCREENSHOT TO INSERT: all 12 length comparisons and path checks (results/compare_status.csv and q7_path_checks.json).

6 Correctness and performance

6.1 Comparison program

compare.py calls run_pg and run_neo, normalizes integers/dates/lists, sorts and compares exactly; Q7 lengths are compared separately from supporting-path validity. In the reference run all 61/61 PostgreSQL results equal the independent pandas answers (reference_answers.py), for both the baseline and improved Q7. Independent CSV checks (independent_checks.py): overlap of 998 and 220998 on device 998 = days [0,29] (pass); Q5 chain A all checks True; Q6 cycle A all checks True.

Question	Cases	PostgreSQL = reference	Neo4j = PostgreSQL
Q1	6	6	<fill>
Q2	1	1	<fill>
Q3	6	6	<fill>
Q4	24	24	<fill>
Q5	6	6	<fill>
Q6	6	6	<fill>
Q7	12	12	<fill>
Total	61	61	<fill>

FILL FROM YOUR NEO4J RUN: run python src/compare.py with both services up; paste the 61-case table; describe and resolve any mismatch.

SCREENSHOT TO INSERT: 61-case status table and the independent CSV checks.

6.2 Timing a fixed selection

Case	PG median ms	PG min	PG max	PG improved median	Neo4j median ms	PG/Neo ratio
Q1-B	0.503	0.298	0.84			
Q1-E	0.273	0.249	0.435			
Q2-month	3453.542	3214.881	3623.448			
Q3-B	1.111	0.853	1.187			
Q3-E	0.432	0.358	0.455			
Q4-B-d10	11.781	9.772	12.862			
Q4-B-d4	2.192	1.837	2.362			
Q4-B-d6	3.291	2.899	4.021			
Q4-B-d8	4.661	4.023	6.925			
Q4-E-d10	10.3	7.192	12.013			
Q4-E-d4	1.221	1.151	1.543			
Q4-E-d6	3.368	3.108	3.654			
Q4-E-d8	6.664	5.205	6.925			

Q5-B	1.194	0.966	1.642			
Q5-E	0.302	0.268	0.663			
Q6-B	0.882	0.654	1.159			
Q6-E	0.329	0.282	0.365			
Q7-B-b4	404.83	319.663	421.192	6.659		
Q7-B-b6	22635.44	21806.872	23049.792	46.59		
Q7-E-b4	277.278	258.12	329.93	6.354		
Q7-E-b6	22757.638	21747.886	23145.063	50.443		

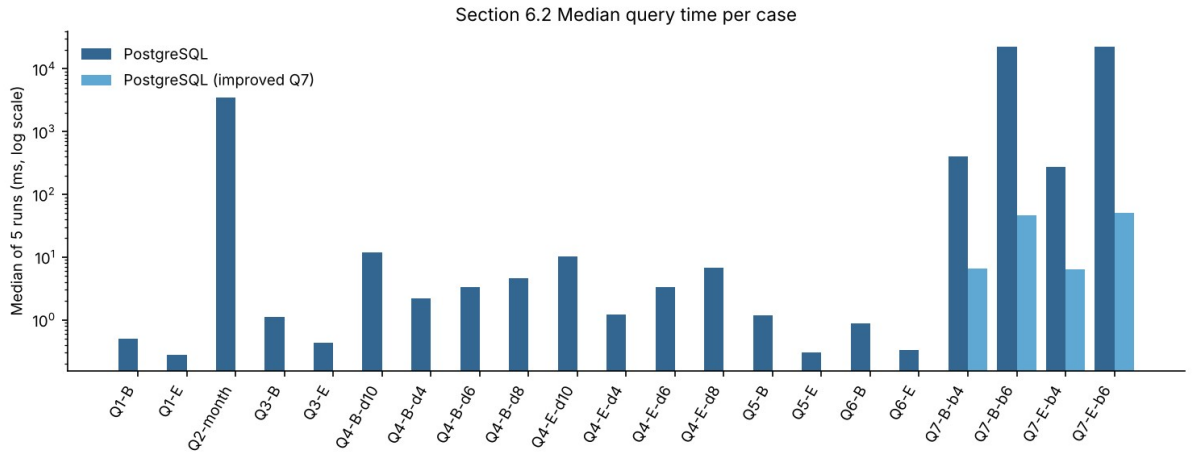


Figure 9 Median query time per case (log scale)

Plans (results/plans): Q2 is a parallel sequential scan of 20 M rows with hash aggregation (≈4.3 s); Q4 B depth 10 runs in 8 ms because the recursive term only touches a few hundred index entries. Baseline Q7 B bound 6: the Recursive Union processes 187,957 frontier rows, fetches 14.6 million transfer index entries and keeps 427,955 rows after UNION de-duplication, 26.8 s in total; the planner estimated 314 rows per iteration against 1.45 million actual. Neo4j's shortestPath instead runs a bidirectional breadth-first search that stops when the frontiers meet.

FILL FROM YOUR NEO4J RUN: 21 Neo4j medians and ratios (benchmark.py --summarize) and PROFILE of Q2, Q4 B d10, Q7 B b6; explain one processing difference.

SCREENSHOT TO INSERT: benchmark summary, EXPLAIN (ANALYZE, BUFFERS) and PROFILE plans.

6.3 One SQL improvement

Wasted work identified from the plan: each frontier account re-reads every parallel transfer (78 per sender on average), accounts already reached are expanded again at later depths, and the search explores 6 levels from one side only. Prediction: searching from both ends over distinct person neighbours (forward ⌈b/2⌉, backward ⌊b/2⌋, minimum df+db where they meet) should cut the explored states from ~16^6 to ~2×16^3 and give a 100× or better speed-up. Result: all 12 Q7 cases still match; B/E bound 4 fell from 405/277 ms to 6.7/6.4 ms and bound 6 from 22,635/22,758 ms to 46.6/50.4 ms (≈450×). The improved version is selected for Section 7 and the final comparison. Both versions are kept (Q7_SQL, Q7_SQL_IMPROVED).

SCREENSHOT TO INSERT: 12-case recheck, the four revised timings and before/after plans (pg_Q7_B_bound6.txt vs pg_Q7_B_bound6_improved.txt).

7 Concurrent requests

7.1 Load test

loadtest.py starts one process per client with its own persistent connection, warms up A-B-C once, synchronizes on a multiprocessing Barrier whose action sets the common start time, starts cycles until start+15 s, finishes the started cycle and logs each request; correctness is checked after timing. Medians of three repetitions:

Engine	Workload	Clients	Throughput req/s	Median ms	p95 ms	Errors	Timeouts	Incorrect
pg	q4d10	1	81.578	5.984	29.991	0	0	0
pg	q4d10	4	153.107	13.261	63.067	0	0	0
pg	q4d10	8	153.738	27.407	125.856	0	0	0
pg	q4d10	16	155.172	52.799	245.078	0	0	0
pg	q7b6	1	23.111	40.517	58.217	0	0	0
pg	q7b6	4	43.444	88.411	122.66	0	0	0
pg	q7b6	8	42.844	181.063	236.044	0	0	0
pg	q7b6	16	41.665	376.208	458.756	0	0	0

FILL FROM YOUR NEO4J RUN: the eight Neo4j combinations (python src/loadtest.py --engine neo, then --summarize).

SCREENSHOT TO INSERT: active client processes during a run and all 16 combination summaries.

7.2 Scaling

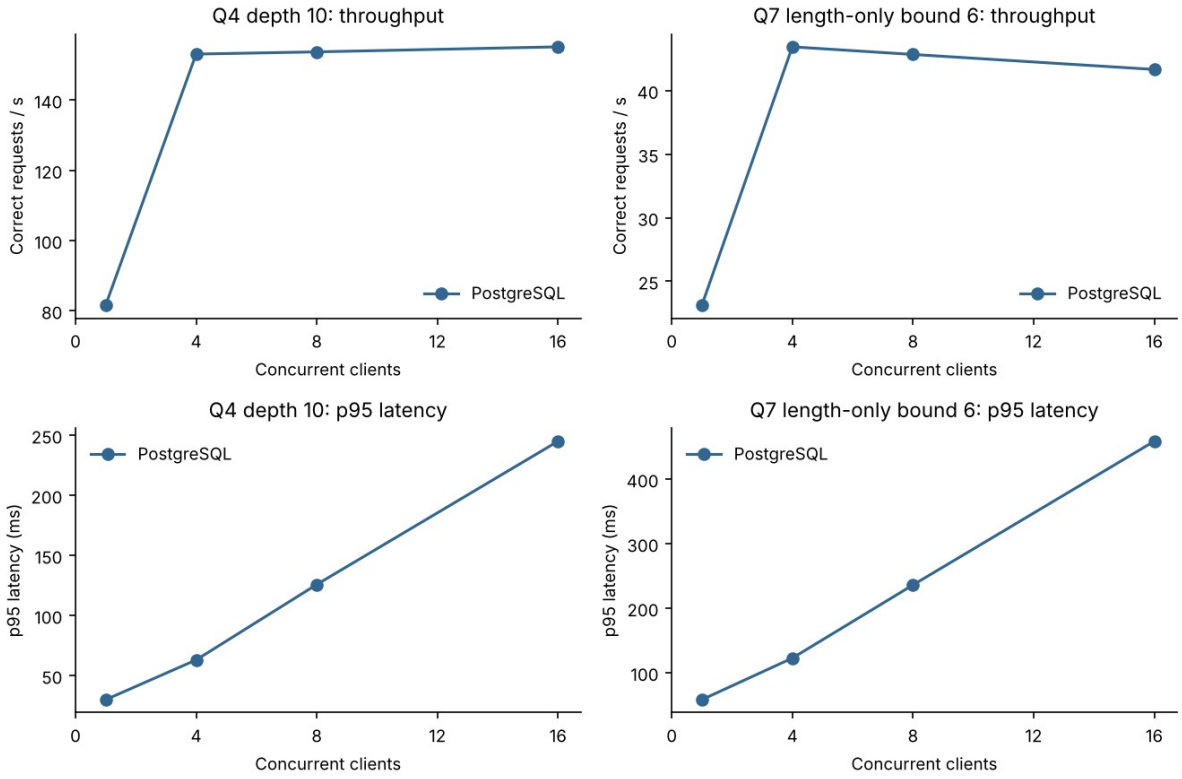


Figure 10 Throughput and p95 against client count

On the 2-vCPU reference machine PostgreSQL throughput rises from 1 to 4 clients (Q4 d10 ≈82 → ≈153 req/s; Q7 b6 ≈23 → ≈44 req/s) and is flat from 4 to 16 clients, while p95 grows roughly in proportion to clients (Q4 ≈30 → ≈245 ms; Q7 ≈58 → ≈459 ms). Both CPUs are saturated, so extra clients only queue: throughput (system capacity) stops improving while

each individual response gets slower. No errors or timeouts occurred and every completion was correct. Repeat on your machine and compare with Neo4j before claiming whether any graph advantage survives the SQL improvement.

FILL FROM YOUR NEO4J RUN: 1 vs 16 client comparison for Neo4j, CPU/memory evidence (Task Manager/htop) and whether a graph advantage remains.

8 Decisions

8.1 Investigation priority

Pair	Q3 devices	Q4 network (10 links)	Q5 / Q6	Q7 (bound 6)	Review first
A / D	Both share a handset with their 3 group members for days 0–29 plus an own-customer wallet	A 187, D 192 accounts	A: 1 chain + 4-transfer cycle (200,000 RWF, 2 min); D: none	A 3, D 3	A
B / E	Both: household phone (5 customers) + group device	B 228, E 230	B: 1 chain of 1,050,000 RWF in 80 s; E: none	B 5, E 5	B
C / F	Both: household phone (one member joined it on day 11 after a phone replacement) + group device	C 1,433, F 1,426	C: 1 chain + 6-transfer cycle (300,000 RWF, 3 min 20 s); F: none	C 5, F 5	C

Detailed pair A/D. Device and network evidence cannot separate them: each shares one handset with exactly its three group members for the whole month and reaches a similar number of accounts. The transfers differ. A sent 200,000 RWF to 8917 at 10:00:00 on 6 Sep; it moved on to 16836 (196,000) and 24755 (192,080) 40 s apart and returned to A as 188,238 RWF at 10:02:00 – a cycle losing 2% per hop. D's group passed 45,000 RWF around the same four-account loop (20001651–20001654) but two days apart (6, 8, 10 and 12 Sep) with equal amounts, which fails the 600 s and 1,800 s windows and looks like a rotating savings group (ikimina) or family support. A is reviewed first.

Ordinary explanation for A: four colleagues using one shop phone might pass a float between them to settle a joint purchase, each keeping a 2% commission. Missing evidence – balances, KYC links between owners, who initiated each payment, what happened to the 188,238 RWF afterwards – could confirm this (raising no concern) or show money returning to its origin after layering (raising priority). These records justify enquiries, not a finding of fraud.

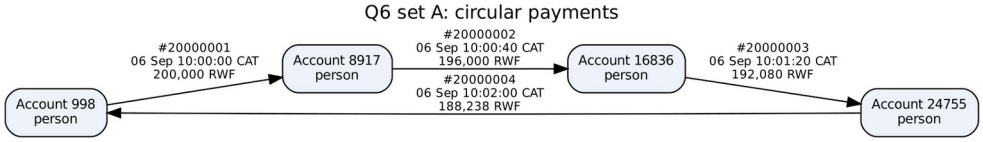


Figure 11 Transfer cycle for A (from Section 5.3)

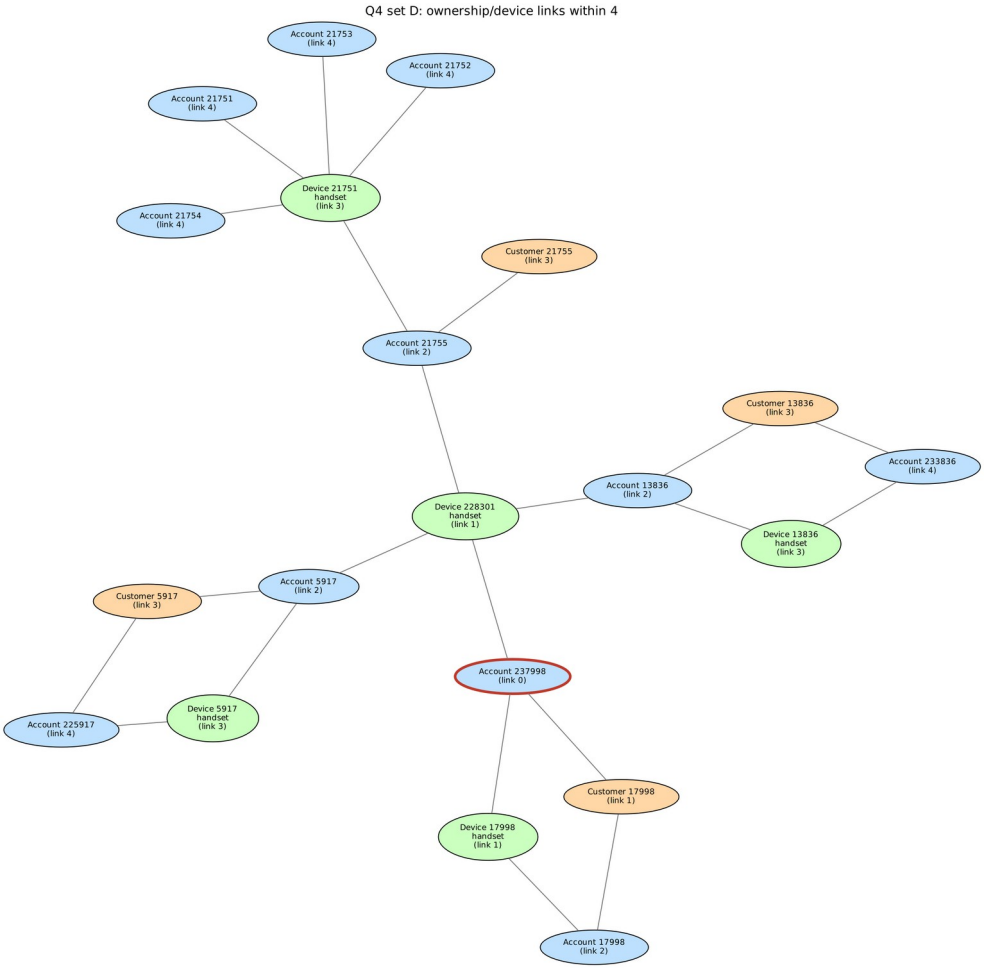


Figure 12 Ownership–device network for D within four links

8.2 Model recommendation (≤500 words)

[Write after your Neo4j results are in. Structure to follow: (1) Reporting (Q1–Q2): evidence from Q2 timings, plan (parallel seq scan + hash aggregate vs relationship scan) and load/storage (PG load 12.9 s + 53.1 s setup, 3.73 GB vs Neo4j import/setup seconds and store GB). (2) Traversal (Q4–Q7): Cypher patterns are shorter and the native shortest path avoided the 22.6 s baseline SQL, but the improved SQL (≈50 ms) shows much of the gap was algorithm, not model – separate graph-modelling benefits (relationships stored as adjacency, readable variable-length patterns) from implementation effects (bidirectional BFS, pruning var-expand). (3) Concurrency: compare saturation points. Include evidence against your choice. Propose a freshness target for copying operational data to the graph (e.g. ≤15 minutes via change-data capture) and the consequence of stale data (missed chains/cycles involving the newest transfers). State two limits: one synthetic month on one machine; no write workload or replication measured.]

8.3 Individual judgement

Each member writes 150–200 words in their own name on a different measured finding, its explanation and a check that could overturn it (e.g. Member 1: Q7 baseline vs improved SQL; Member 2: load-test saturation at 4 clients; Member 3: Q4 growth with depth). Do not reuse text from this draft.

Member	Contributions	LLM / external assistance and how it was checked
<Name 1>		
<Name 2>		
<Name 3>		e.g. Claude (Anthropic) used to draft code and this report skeleton; every query re-run,

		answers verified against independent pandas results and CSV rows; Neo4j queries tested by the group.

9 Reproducibility

README.md lists every command in execution order; .env.example holds placeholder secrets; CSVs and database stores stay local. verify_generator.py confirms the generator SHA-256 2ea0e59d…455a is unchanged.

SCREENSHOT TO INSERT: final rerun of python src/compare.py (61 cases complete) and both services stopped (pg_ctl status / neo4j status).